BACKUP AIRLINES

Informe Técnico Completo del Sistema

Actividad Grupal — Seguridad en Aplicaciones Web

Autenticación · Autorización · Sesiones · Manejo de Errores · CRUD

Spring Boot 3 · Spring Security 6 · H2 Database · Thymeleaf 3 · Bootstrap 5

Abril 2026

1. Descripción General del Proyecto

Backup Airlines es un sistema web de gestión de archivos construido con Spring Boot 3 y Spring Security 6, utilizando H2 como base de datos relacional en memoria. El sistema permite a usuarios autenticados subir, gestionar, compartir y versionar archivos, con un panel administrativo para la gestión de usuarios y monitoreo de actividad.

Proyecto	Backup Airlines — Sistema de Gestión de Archivos
Tecnología principal	Java 17 / Spring Boot 3 / Spring Security 6
Base de datos	H2 (en memoria) con Hibernate ORM / JPA
Motor de vistas	Thymeleaf 3 + Bootstrap 5 + Bootstrap Icons
Almacenamiento físico	Sistema de archivos local (carpeta uploads/)
Puerto	8080 (configurable en application.properties)
Autenticación	DaoAuthenticationProvider + BCryptPasswordEncoder
Gestión de sesiones	Spring Security Session Management (máx. 1 por usuario)

2. Arquitectura del Proyecto

El proyecto sigue una arquitectura MVC (Model-View-Controller) estándar de Spring Boot, organizada en capas bien definidas:

2.1 Estructura de paquetes

- com.example.backup.model — Entidades JPA (@Entity)
- com.example.backup.repository — Interfaces JpaRepository (acceso a datos)
- com.example.backup.service — Lógica de negocio (@Service)
- com.example.backup.controller — Controladores web (@Controller)
- com.example.backup.config — Configuración de seguridad (@Configuration)
- com.example.backup.exception — Manejo global de errores (@ControllerAdvice)

2.2 Flujo de una petición

- El navegador envía una petición HTTP al servidor en el puerto 8080.
- Spring Security intercepta la petición y verifica autenticación y roles antes de que llegue al controlador.
- El controlador llama al servicio correspondiente, quien interactúa con los repositorios JPA.
- JPA/Hibernate traduce las operaciones Java a SQL y las ejecuta en H2.
- El controlador devuelve el nombre de la vista; Thymeleaf la renderiza con los datos del Model.

 La base de datos H2 es en memoria (jdbc:h2:mem:backupdb). Todos los datos se pierden al reiniciar la aplicación. Para persistencia permanente se puede cambiar a jdbc:h2:file:./data/backupdb en application.properties.

3. Base de Datos — Tablas y Relaciones

Hibernate genera automáticamente todas las tablas al arrancar la aplicación (ddl-auto=update). A continuación se describen las 10 tablas que componen el esquema:

Tabla	Columnas principales	Relaciones FK
users	id, username (UNIQUE), password, fullName, enabled	→ users_roles
roles	id, name (UNIQUE)	← users_roles
users_roles	user_id, role_id	FK user_id → users, FK role_id → roles
backup_files	id, filename, storageFilename, contentType, size, createdAt, trashed, trashedAt, favorite	FK owner_id → users, FK deleted_by → users, FK folder_id → folders
file_versions	id, filename, storageFilename, contentType, size, createdAt	FK backup_file_id → backup_files
folders	id, name, createdAt	FK user_id → users, FK parent_id → folders (auto-ref)
file_shares	id, publicToken, permission (READ/WRITE), expiresAt	FK file_id → backup_files, FK shared_with_user_id → users
tags	id, name (UNIQUE, max 50 chars)	← file_tags
file_tags	file_id, tag_id (clave compuesta PK)	FK file_id → backup_files, FK tag_id → tags
activity_logs	id, action (VARCHAR), createdAt	FK user_id → users (nullable), FK file_id → backup_files (nullable)

3.1 Relaciones clave

- users ←→ roles: relación ManyToMany a través de la tabla intermedia users_roles.
- backup_files → users (owner_id): cada archivo pertenece a un usuario.
- backup_files → users (deleted_by): registra quién realizó la eliminación lógica.
- backup_files → folders: organización en carpetas (opcional, nullable).
- backup_files ←→ tags: ManyToMany a través de file_tags (clave compuesta PK).
- file_versions → backup_files: historial de versiones por archivo.
- file_shares → backup_files + users: compartición entre usuarios o por token público.
- activity_logs → backup_files + users: ambas FK son nullable para sobrevivir deletes.
- folders → folders (parent_id): auto-referencia para soporte de subcarpetas.

3.2 Datos iniciales (seed al arrancar)

BackupApplication.java ejecuta un CommandLineRunner que siembra:

- Rol ROLE_ADMIN y ROLE_USER (si no existen)
- Usuario admin / admin123 con ROLE_ADMIN
- Usuario usuario1 / user1234 con ROLE_USER
- Tags de ejemplo: importante, trabajo, personal, urgente

3.3 Cómo acceder a la consola H2

```
URL: http://localhost:8080/h2-console
JDBC URL: jdbc:h2:mem:backupdb
Usuario: sa    Contraseña: (vacía)
```

4. Inventario Completo de Archivos

La siguiente tabla detalla todos los archivos del proyecto, indicando si son nuevos, modificados o ya existían, junto con el descripción de los cambios:

Archivo	Estado	Cambios realizados
BackupFile.java	**MODIFICADO**	Agregados: favorite, trashedAt, deletedBy (FK→User), folder (FK→Folder), tags (ManyToMany), shares (OneToMany). Métodos helper de formato de fecha.
FileVersion.java	**MODIFICADO**	Agregado: getCreatedAtFormatted() usando ZoneId America/Bogota.
User.java	**EXISTÍA**	Sin cambios estructurales. Usado en nuevas relaciones.
Role.java	**EXISTÍA**	Sin cambios.
Folder.java	**NUEVO**	Entidad para carpetas con soporte de subcarpetas (parent_id auto-referenciado). Relación ManyToOne → User.
Tag.java	**NUEVO**	Etiquetas con nombre único. ManyToMany con BackupFile.
FileShare.java	**NUEVO**	Compartición de archivos por usuario o token público. Enum Permission (READ/WRITE), campo expiresAt.
ActivityLog.java	**NUEVO**	Registro de acciones (UPLOAD_FILE, DELETE_FILE, SHARE_FILE, DOWNLOAD_FILE, etc.). FK nullable a User y BackupFile. Método getCreatedAtFormatted().
BackupFileRepository	**MODIFICADO**	Nuevos métodos: findByOwnerIdAndTrashedFalse, findByOwnerIdAndTrashedTrue, findByTrashedTrue.
ActivityLogRepository	**MODIFICADO**	Agregados: findByFileId(), findTop50ByOrderByCreatedAtDesc(), @Modifying nullifyFileReference() para evitar FK violation al eliminar archivos.
FolderRepository	**NUEVO**	findByOwnerIdAndParentIsNull() — carpetas raíz del usuario.
TagRepository	**NUEVO**	findByName() para reutilizar tags existentes.
FileShareRepository	**NUEVO**	findBySharedWithUserId(), findByPublicToken(), findByFileId().
FileStorageService.java	**MODIFICADO**	permanentDelete() ahora llama nullifyFileReference() antes de borrar (fix FK violation). store() detecta versiones existentes. Inyectado ActivityLogRepository.
ActivityLogService.java	**NUEVO**	Servicio centralizado: log(user, action, file) y getRecent() (últimas 50 entradas).

Archivo	Estado	Cambios realizados
CleanupService.java	EXISTÍA	Sin cambios. Ejecuta purga automática a las 03:00 AM.
UserService.java	EXISTÍA	Sin cambios. Implementa UserDetailsService para Spring Security.
FileController.java	MODIFICADO	17 endpoints. Nuevos: favorito toggle, compartir (usuario/público), carpetas, tags, papelera mejorada, enlace público /public/{token}, registro de actividad en cada operación.
AdminController.java	MODIFICADO	CRUD completo de usuarios: crear con rol, activar/desactivar, eliminar. Vista de logs. Eliminar cualquier archivo.
AuthController.java	EXISTÍA	Sin cambios estructurales. Registro asigna ROLE_USER.
CustomErrorController	NUEVO	Implementa ErrorController. Mapea /error con código HTTP real. Reemplaza Whitelabel Error Page.
SecurityConfig.java	MODIFICADO	Agregados: HttpSessionEventPublisher (para maximumSessions), /error como ruta pública, ruta /admin/** con hasRole(ADMIN), sessionManagement con timeout y expiredUrl.
GlobalExceptionHandler	NUEVO	@ControllerAdvice con handlers para SecurityException (403), FileNotFoundException (404), MaxUploadSizeExceededException (redirect), Exception genérica (500).
dashboard.html	MODIFICADO	Secciones: subida múltiple, selector de carpeta, tabla de archivos con favorito/compartir/versiones/papelera, archivos compartidos conmigo, modal de compartir por usuario o token público.
login.html	MODIFICADO	Mensajes de error/logout/expiración de sesión. Diseño Bootstrap con gradiente.
register.html	EXISTÍA	Sin cambios.
trash.html	MODIFICADO	Muestra trashedAtFormatted, deletedBy. Botones restaurar y eliminar permanentemente.
file-versions.html	MODIFICADO	Usa createdAtFormatted (sin #temporals que no existe en Thymeleaf 3).
admin-users.html	MODIFICADO	Formulario de creación de usuario con selector de rol. Toggle de estado. Botón eliminar.
admin-files.html	MODIFICADO	Muestra favorite, trashed. Botón de eliminación permanente.
admin-logs.html	NUEVO	Tabla de actividad reciente (50 registros) con código de color por acción.
error.html	NUEVO	Página de error personalizada con código HTTP, mensaje descriptivo y botón de regreso.

5. Endpoints del Sistema (API REST / MVC)

El sistema expone 29 endpoints. Los niveles de acceso son: Público (sin autenticación), Auth (usuario autenticado) y ADMIN (requiere ROLE_ADMIN).

Método	Ruta	Acceso	Descripción
GET	/login	Público	Página de inicio de sesión
POST	/login	Público	Procesar credenciales (Spring Security)
GET	/register	Público	Formulario de registro
POST	/register	Público	Crear cuenta con ROLE_USER
GET	/logout	Auth	Cerrar sesión e invalidar cookie
GET	/ o /dashboard	Auth	Panel principal con archivos, carpetas y compartidos
POST	/upload	Auth	Subir uno o varios archivos (multipart, máx 50 MB)
GET	/file/{id}/download	Auth	Descargar archivo (verifica propiedad o compartición)
GET	/file/{id}/preview	Auth	Visualizar inline en navegador
POST	/file/{id}/favorite	Auth	Toggle favorito (activo/inactivo)
POST	/file/{id}/trash	Auth	Eliminación lógica (soft delete)
GET	/trash	Auth	Vista de papelera del usuario
POST	/file/{id}/restore	Auth	Restaurar archivo de la papelera
POST	/file/{id}/delete	Auth	Eliminación permanente (disco + BD)
GET	/file/{id}/versions	Auth	Historial de versiones del archivo
POST	/file/{fid}/version/{vid}/restore	Auth	Restaurar versión anterior como nueva versión
POST	/file/{id}/share	Auth	Compartir con usuario o generar token público
GET	/public/{token}	Público	Descargar archivo por enlace público (verifica expiración)
POST	/folder/create	Auth	Crear carpeta (opcional: parent_id para subcarpeta)
POST	/file/{id}/tag	Auth	Agregar etiqueta al archivo
GET	/admin/users	ADMIN	Listar todos los usuarios
POST	/admin/user/create	ADMIN	Crear usuario con rol específico
POST	/admin/user/{id}/toggle	ADMIN	Activar / desactivar usuario
POST	/admin/user/{id}/delete	ADMIN	Eliminar usuario
GET	/admin/files	ADMIN	Ver todos los archivos del sistema
POST	/admin/file/{id}/delete	ADMIN	Eliminar permanentemente cualquier archivo
GET	/admin/logs	ADMIN	Últimas 50 acciones registradas

Método	Ruta	Acceso	Descripción
GET	/h2-console/**	Público*	Consola web de H2 (solo en desarrollo)
GET	/error	Público	Página de error personalizada (reemplaza Whitelabel)

6. Seguridad — Autenticación, Autorización y Sesiones

6.1 Autenticación

Se usa DaoAuthenticationProvider configurado con UserDetailsService (UserService.java) y BCryptPasswordEncoder. El UserService carga el usuario desde H2 por username y convierte sus roles en GrantedAuthority de Spring Security.

6.2 Autorización por roles

- ROLE_USER: acceso a dashboard, subida, descarga, papelera, versiones, compartir, favoritos.
- ROLE_ADMIN: acceso adicional a /admin/users, /admin/files, /admin/logs.
- Cualquier intento de acceso sin permisos retorna HTTP 403 (manejado por GlobalExceptionHandler).

6.3 Gestión de sesiones

- maximumSessions(1): solo una sesión activa por usuario simultáneamente.
- maxSessionsPreventsLogin(false): la nueva sesión desplaza a la anterior.
- expiredUrl('/login?expired'): sesión desplazada redirige al login con mensaje.
- Timeout de 30 minutos configurado en application.properties.
- Logout invalida la sesión HTTP y elimina la cookie JSESSIONID.
- HttpSessionEventPublisher bean requerido para que maximumSessions funcione.

6.4 Rutas públicas

Las siguientes rutas son accesibles sin autenticación: /login, /register, /logout, /error, /public/{token} (descarga por enlace), /h2-console/**, recursos estáticos (/css/**, /js/**, /images/**).

7. Manejo de Errores

El sistema tiene una cadena de manejo de errores en dos niveles:

7.1 GlobalExceptionHandler (@ControllerAdvice)

Intercepta excepciones lanzadas dentro de los controladores antes de que lleguen al cliente:

- SecurityException → HTTP 403 → vista error.html
- FileNotFoundException / NoSuchFileException → HTTP 404 → vista error.html
- MaxUploadSizeExceededException → redirect /dashboard con mensaje flash
- Exception (genérica) → HTTP 500 → vista error.html con el mensaje del error

7.2 CustomErrorController

Implementa ErrorController de Spring Boot. Mapea /error y lee el código HTTP real del RequestDispatcher. Reemplaza completamente la página Whitelabel de Spring Boot. Muestra mensajes amigables para 403, 404 y 500.

 La ruta /error debe estar en la lista de rutas permitidas (permitAll) en SecurityConfig, de lo contrario Spring Security intercepta la renderización de la página de error antes de llegar al controlador.

8. Bugs Identificados y Corregidos

Bug	Causa raíz	Solución aplicada
Whitelabel Error Page en /error	Spring Boot usa /error como fallback, pero ningún @Controller lo mapeaba. Thymeleaf no renderizaba error.html.	Creado CustomErrorController que implementa ErrorController. Mapea /error y lee el código HTTP real del request.
#temporals no disponible en Thymeleaf 3	El objeto #temporals pertenece a thymeleaf-extras-java8time, no incluido en el pom.xml. El template explotaba en tiempo de render.	Reemplazado por métodos Java getCreatedAtFormatted() / getTrashedAtFormatted() en los modelos usando DateTimeFormatter con ZoneId America/Bogota.
FK violation al eliminar archivo permanentemente	activity_logs.file_id tiene FK hacia backup_files. Al intentar DELETE sin limpiar los logs, H2 lanzaba Referential integrity constraint violation [23503-214].	Agregado @Modifying nullifyFileReference() en ActivityLogRepository que hace UPDATE masivo poniendo file_id = null antes del delete.
maximumSessions(1) no funcionaba	Spring Security necesita un HttpSessionEventPublisher bean para detectar destrucción de sesiones y controlar el límite.	Agregado @Bean HttpSessionEventPublisher en SecurityConfig.
findByFileId() no declarado en repositorio	FileStorageService llamaba activityLogRepository.findByFileId() pero el método no existía en la interfaz, causando NoSuchMethodException.	Agregado findByFileId(Long fileId) a ActivityLogRepository y reemplazada la iteración por la query @Modifying.

9. Cumplimiento de Requisitos de la Actividad

Req.	Requisito de la actividad	Implementación	Estado
a	**Autenticación en BD con mínimo 2 roles**	UserDetailsService + DaoAuthenticationProvider contra H2. Roles ROLE_ADMIN y ROLE_USER seedeados al arrancar. Registro público asigna ROLE_USER.	☑OK
b	**Definición de privilegios, roles y restricción de rutas**	/admin/** solo con hasRole(ADMIN). Cualquier otra ruta requiere autenticación. SecurityException lanza 403. Acceso denegado a archivos de otros usuarios.	☑OK
c	**Formulario de login y loginPage**	loginPage('/login') en SecurityConfig. Template login.html con mensajes de error (?error), logout (?logout) y sesión expirada (?expired). failureUrl configurado.	☑OK
d	**Administración de sesiones y tiempo de vida**	maximumSessions(1) + HttpSessionEventPublisher.	☑OK

Req.	Requisito de la actividad	Implementación	Estado
		server.servlet.session.timeout=30m. expiredUrl='/login?expired'. Logout invalida sesión y borra JSESSIONID.	
e	**Manejo de control de errores**	@ControllerAdvice GlobalExceptionHandler (403, 404, 500, MaxUpload). CustomErrorController reemplaza Whitelabel. Página error.html personalizada.	☑OK
f	**Backend con CRUD demostrable por roles**	10 entidades @Entity en H2. 29 endpoints REST/MVC. CRUD completo de usuarios (admin), archivos, carpetas, tags, shares y logs. Verificación de propiedad en cada operación.	☑OK

10. Instrucciones de Configuración y Ejecución

10.1 Requisitos previos

- Java 17 o superior instalado (java -version para verificar)
- Maven 3.8+ (mvn -version) o usar el wrapper ./mvnw incluido
- No se requiere base de datos externa — H2 se levanta automáticamente

10.2 Ejecución

```
# 1. Descomprime el proyecto
unzip backup-airlines-completo.zip
cd backup-airlines-entrega

# 2. Compila y ejecuta
mvn spring-boot:run

# O si tienes el wrapper Maven:
./mvnw spring-boot:run
```

10.3 Acceso inicial

- Dashboard: http://localhost:8080/dashboard
- Login: http://localhost:8080/login
- H2 Console: http://localhost:8080/h2-console
- Admin: http://localhost:8080/admin/users (solo usuario admin)

10.4 Accesos rápidos

URL	Descripción
`http://localhost:8080/login`	Acceso
`http://localhost:8080/dashboard`	Panel principal
`http://localhost:8080/h2-console`	Base de datos H2
`http://localhost:8080/admin/users`	Gestión de usuarios (solo admin)
`http://localhost:8080/admin/logs`	Registro de actividad (solo admin)

10.5 Usuarios por defecto

Usuario	Contraseña	Rol
admin	admin123	ROLE_ADMIN — acceso a panel de administración y a todo el sistema
usuario1	user1234	ROLE_USER — acceso al dashboard, subida y gestión de sus propios archivos

10.6 Cambiar a H2 persistente (opcional)

Si se desea que los datos sobrevivan al reinicio, cambiar en application.properties:

```
# Cambiar esta línea:
spring.datasource.url=jdbc:h2:mem:backupdb;DB_CLOSE_DELAY=-1

# Por esta:
spring.datasource.url=jdbc:h2:file:./data/backupdb
```

☑ Con la configuración de archivo, H2 creará la carpeta data/ en la raíz del proyecto y guardará backupdb.mv.db con todos los datos de forma persistente entre reinicios.

11. Notas Adicionales

Limpieza automática de papelera

CleanupService ejecuta un @Scheduled(cron = '0 0 3 * * ?') diariamente a las 3:00 AM. Elimina permanentemente los archivos en papelera con más de 30 días (configurable con app.trash.retention-days en application.properties).

Versionado de archivos

Cada subida crea un registro en file_versions. Si el usuario sube un archivo con el mismo nombre que uno existente (no borrado), se crea una nueva versión del mismo en lugar de duplicar el registro. Desde /file/{id}/versions se puede restaurar cualquier versión anterior.

Compartición de archivos

Un archivo puede compartirse de dos formas: (1) con un usuario específico del sistema (por username), o (2) mediante un enlace público con token UUID que expira en 7 días. Los archivos compartidos aparecen en la sección 'Compartidos conmigo' del dashboard del destinatario.

 El frontend (Thymeleaf + Bootstrap 5) no es evaluado en la actividad, pero está completamente implementado con diseño responsivo, mensajes flash, modales de compartir, e indicadores de favorito y estado de papelera.